



Chapter 3.4

Self-Organising Networks



Introduction

- In the previous chapters we discussed a number of networks.
- Training is given to the network.
- Training patterns are of the form of $(x^p; d^p)$
- Mapping is done in such a manner so that $d^p = F(x^p)$
- Training is done with the presence of an external teacher
- Supervised approach.
- Sometimes such training data, consisting of input and desired output pairs are not available
- Only a set of input patterns x^p is available

Some examples of such problems



- **Clustering**: the input data may be grouped in 'clusters' and the data processing system has to find these inherent clusters in the input data. The output of the system should give the cluster label of the input pattern (discrete output);
- **Vector quantization**: this problem occurs when a continuous space has to be discretized. The input of the system is the n -dimensional vector x , the output is a discrete representation of the input space. The system has to find optimal discretization of the input space;
- **Dimensionality reduction**: the input data are grouped in a subspace which has lower dimensionality than the dimensionality of the data. The system has to learn an optimal mapping, such that most of the variance in the input data is preserved in the output data;
- **Feature extraction**: the system has to extract features from the input signal. This often means a dimensionality reduction as described above.

Neuro-computational approaches



- Number of neuro-computational approaches are available to solve these kinds of problems.
- Training is done **without the presence of an external teacher**.
- The **unsupervised** weight adapting algorithms are usually based on some form of global competition between the neurons.



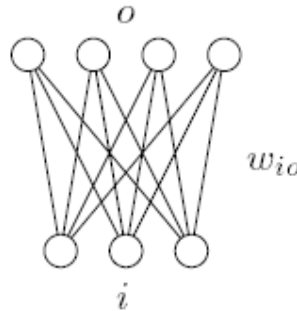
Types

- Competitive learning as proposed by Rumelhart and Zipser (Rumelhart & Zipser, 1985)
- A very similar network but with different emergent properties is the topology-conserving map devised by Kohonen
- Other self-organising networks are ART, proposed by Carpenter and Grossberg (Carpenter & Grossberg, 1987a; Grossberg, 1976), and Fukushima's cognitron (Fukushima, 1975, 1988).



Clustering

- Competitive learning is a learning procedure that divides a set of input patterns in clusters that are inherent to the input data.
- A competitive learning network is provided only with input vectors x and thus implements an **unsupervised learning** procedure.



- An example of a competitive learning network
- All output units o are connected to all input units i with weights w_{io}
- When an input pattern x is presented, only a single output unit of the network (the winner) will be activated



Clustering(Cont.)

- In a correctly trained network, all x in one cluster will have the same winner
- Two methods are
 1. Dot product
 2. Euclidean distance



Dot product

- **Assumptions:** Both input vectors $\mathbf{x}$ and weight vectors $\mathbf{w}_o$ are normalized to unit length
- Each output unit o calculates its activation value y_o according to the dot product of input and weight vector

$$y_o = \sum_i w_{io} x_i = \mathbf{w}_o^T \mathbf{x}.$$

- In a next pass, output neuron k is selected with maximum activation

$$\forall o \neq k : \quad y_o \leq y_k.$$

- Activations are reset such that $y_k = 1$ and $y_{o \neq k} = 0$.
- Output layer is referred as the **winner-take-all** layer.
- Winner-take-all layer is usually implemented in software by simply selecting the output neuron with highest activation value.



Dot product(Cont.)

- Once the winner k has been selected, the weights are updated according to

$$\mathbf{w}_k(t+1) = \frac{\mathbf{w}_k(t) + \gamma(\mathbf{x}(t) - \mathbf{w}_k(t))}{\|\mathbf{w}_k(t) + \gamma(\mathbf{x}(t) - \mathbf{w}_k(t))\|}$$

- Note that only the weights of winner k are updated.



Euclidean distance

- Previous “Dot product” algorithm would fail if unnormalised vectors are used.
- Naturally one would like to accommodate the algorithm for unnormalised input data.
- In Euclidean distance algorithm winning neuron k is selected with its weight vector w_k closest to the input pattern x , using the Euclidean distance measure

$$k : \|w_k - x\| \leq \|w_o - x\| \quad \forall o.$$

- The weight update must be changed to implement a shift towards the input.

$$w_k(t+1) = w_k(t) + \gamma(x(t) - w_k(t)).$$

- Only the weights of the winner are updated.

Euclidean distance(Cont.)



- A point of attention in these recursive clustering techniques is the initialization
- Especially if the input vectors are drawn from a large or high-dimensional input space, it is not beyond imagination that a **randomly** initialized weight vector w_0 will never be chosen as the winner and will thus never be moved and never be used.
- Therefore, it is customary to initialize weight vectors to a set of input patterns $\{x\}$ drawn from the input set at random.
- Another more thorough approach that avoids these and other problems in competitive learning is called leaky learning.

$$w_l(t+1) = w_l(t) + \gamma'(x(t) - w_l(t)) \quad \forall l \neq k$$

- with $\gamma' \ll \gamma$ the leaky learning rate.



Euclidean distance(Cont.)

In this algorithm,

- Each neuron records the number of times it is selected winner.
- The more often it wins, the less sensitive it becomes to competition.
- Conversely, neurons that consistently fail to win increase their chances of being selected winner.



Cost function

- Competitive network performs a clustering process on the input data
- Input patterns are divided in disjoint clusters such that similarities between input patterns in the same cluster are much bigger than similarities between inputs in different clusters.
- A common criterion to measure the quality of a given clustering is the square error criterion.

$$E = \sum_p \|\mathbf{w}_k - \mathbf{x}^p\|^2$$

where k is the winning neuron when input $\mathbf{x}^p$ is presented.

- The weights $\mathbf{w}$ are interpreted as cluster centres.
- Competitive learning indeed seeks to find a minimum for this square error by following the negative gradient of the error-function.



Vector quantization

- A vector quantization scheme divides the input space in a number of **disjoint** subspaces.
- It represents each input vector x by the label of the subspace it falls into.
- The **difference with clustering** is that we are not so much interested in finding clusters of similar data, but more in quantizing the entire input space.
- The quantization performed by the competitive learning network is said to 'track the input probability density function'.
- The density of neurons and thus subspaces is highest in those areas where inputs are most likely to appear.
- More coarse quantization is obtained in those areas where inputs are scarce



Vector quantization(Cont.)

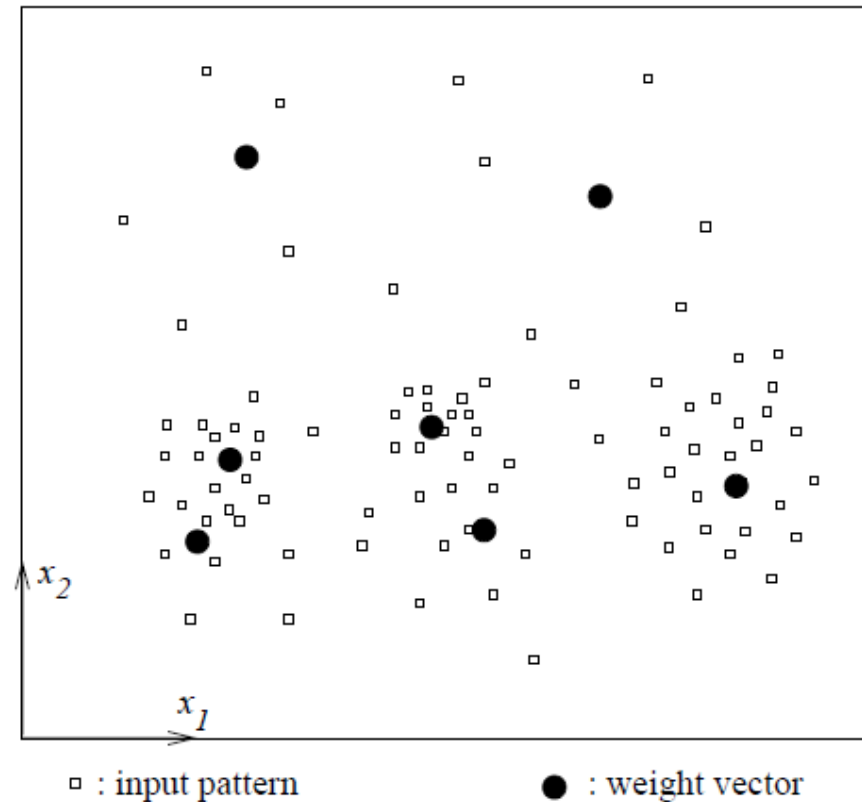


Figure 6.5: This figure visualises the tracking of the input density. The input patterns are drawn from $\mathbb{R}^2$; the weight vectors also lie in $\mathbb{R}^2$. In the areas where inputs are scarce, the upper part of the figure, only few (in this case two) neurons are used to discretise the input space. Thus, the upper part of the input space is divided into two large separate regions. The lower part, however, where many more inputs have occurred, five neurons discretise the input space into five smaller subspaces.



Vector quantization(Cont.)

- Competitive learning can be used for data compression
- For example: telecommunication or storage.
- Competitive learning can be used in combination with supervised learning methods.
- For example: Function approximation problems or classification problems
- Methods are “counterpropagation” and “learning vector quantization”



Counterpropagation

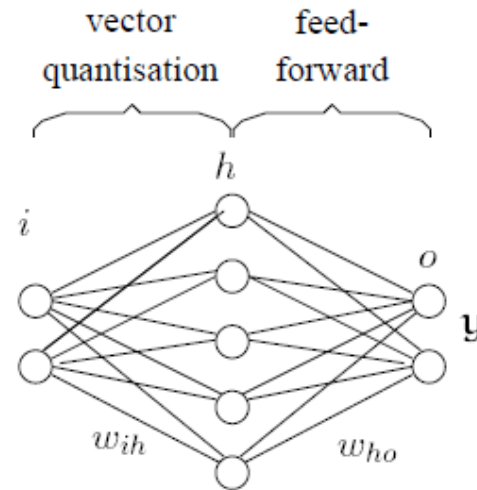


Figure 6.6: A network combining a vector quantisation layer with a 1-layer feed-forward neural network. This network can be used to approximate functions from $\mathbb{R}^2$ to $\mathbb{R}^2$, the input space $\mathbb{R}^2$ is discretised in 5 disjoint subspaces.



Counterpropagation(Cont.)

- This network can approximate a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$
- With each neuron o a function value is associated $[w_{1o}, w_{2o}, \dots, w_{mo}]^T$
- The function value $[w_{1o}, w_{2o}, \dots, w_{mo}]^T$ in entry is taken as an approximation of $f(x)$
- The network can be supervisedly trained in the following way
 1. Present the network with both input x and function value $d = f(x)$;
 2. Perform the unsupervised quantization step. For each weight vector, calculate the distance from its weight vector to the input pattern and find winner k . Update the weights with equation

$$w_k(t+1) = w_k(t) + \gamma(x(t) - w_k(t)).$$

3. Perform the supervised approximation step:

$$w_{ko}(t+1) = w_{ko}(t) + \gamma(d_o - w_{ko}(t)).$$

This is simply the δ -rule with $y_o = \sum_h y_h w_{ho} = w_{ko}$ when k is the winning neuron and the desired output is given by $d = f(x)$.

Learning Vector Quantisation(LVQ)



■ Basic algorithm

1. With each output neuron o , a class label (or decision of some other kind) y_o is associated;
2. A learning sample consists of input vector x^p together with its correct class label $y^p o$;
3. Distance measures between weight vectors w^o and input vector x^p , not only the winner k_1 is determined, but also the second best k_2 :
$$\|x^p - w_{k_1}\| < \|x^p - w_{k_2}\| < \|x^p - w_i\| \quad \forall o \neq k_1, k_2;$$
4. The labels $y^p_{k_1}$, $y^p_{k_2}$ are compared with d^p . The weight update rule is used selectively based on this comparison.

Learning Vector Quantisation(LVQ) (Cont.)



- Last step is given by the LVQ algorithm by Kohonen (Kohonen, 1977),
- If $y_{k_1}^p \neq d^p$ and $d^p = y_{k_2}^p$
- And $\|\mathbf{x}^p - \mathbf{w}_{k_2}\| - \|\mathbf{x}^p - \mathbf{w}_{k_1}\| < \epsilon$;
- Then $\mathbf{w}_{k_2}(t+1) = \mathbf{w}_{k_2} + \gamma(\mathbf{x} - \mathbf{w}_{k_2}(t))$
- And $\mathbf{w}_{k_1}(t+1) = \mathbf{w}_{k_1}(t) - \gamma(\mathbf{x} - \mathbf{w}_{k_1}(t))$
- $\mathbf{w}_{k_2}$ with the correct label is moved towards the input vector, while $\mathbf{w}_{k_1}$ with the incorrect label is moved away from it.



Kohonen network

- The Kohonen network (Kohonen, 1982, 1984) can be seen as an extension to the competitive learning network, although this is chronologically incorrect.
- The output units in S are ordered in some fashion.
- When learning patterns are presented to the network, the weights to the output units are thus adapted such that the order present in the input space $\mathcal{R}^N$ is preserved in the output, i.e., the neurons in S .
- This means that learning patterns which are near to each other in the input space (where 'near' is determined by the distance measure used in finding the winning unit) must be mapped on output units which are also near to each other, i.e., the same or neighbouring units.
- The mapping, which represents a discretisation of the input space, is said to be **topology preserving**.



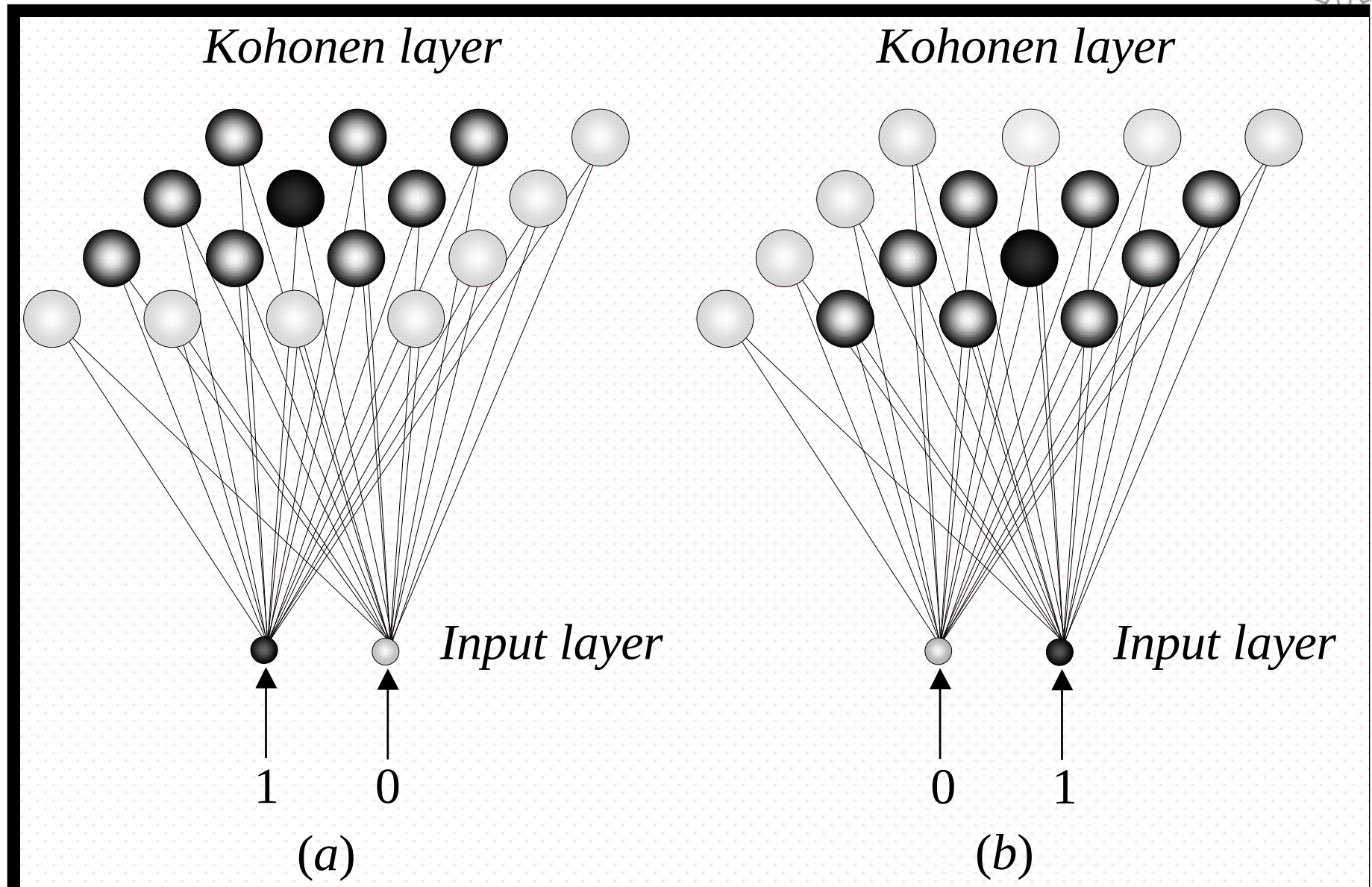
Kohonen network(Cont.)

- Dimensionality of S must be at least N .
- For example: data on a two dimensional manifold in a high dimensional input space can be mapped onto a two-dimensional Kohonen network.
- The learning patterns are random samples from $\mathbb{R}^N$
- At time t , a sample $x(t)$ is generated and presented to the network.
- The winning unit k is determined
- The weights to this winning unit as well as its neighbours are adapted using the learning rule

$$w_o(t+1) = w_o(t) + \gamma g(o, k) (x(t) - w_o(t)) \quad \forall o \in S.$$

- Here, $g(o, k)$ is a decreasing function of the grid-distance between units o and k , such that $g(k, k) = 1$.
- For example, for $g()$ a Gaussian function can be used, such that (in one dimension!) $g(o, k) = \exp(-(o-k)^2)$

Feature-mapping Kohonen model

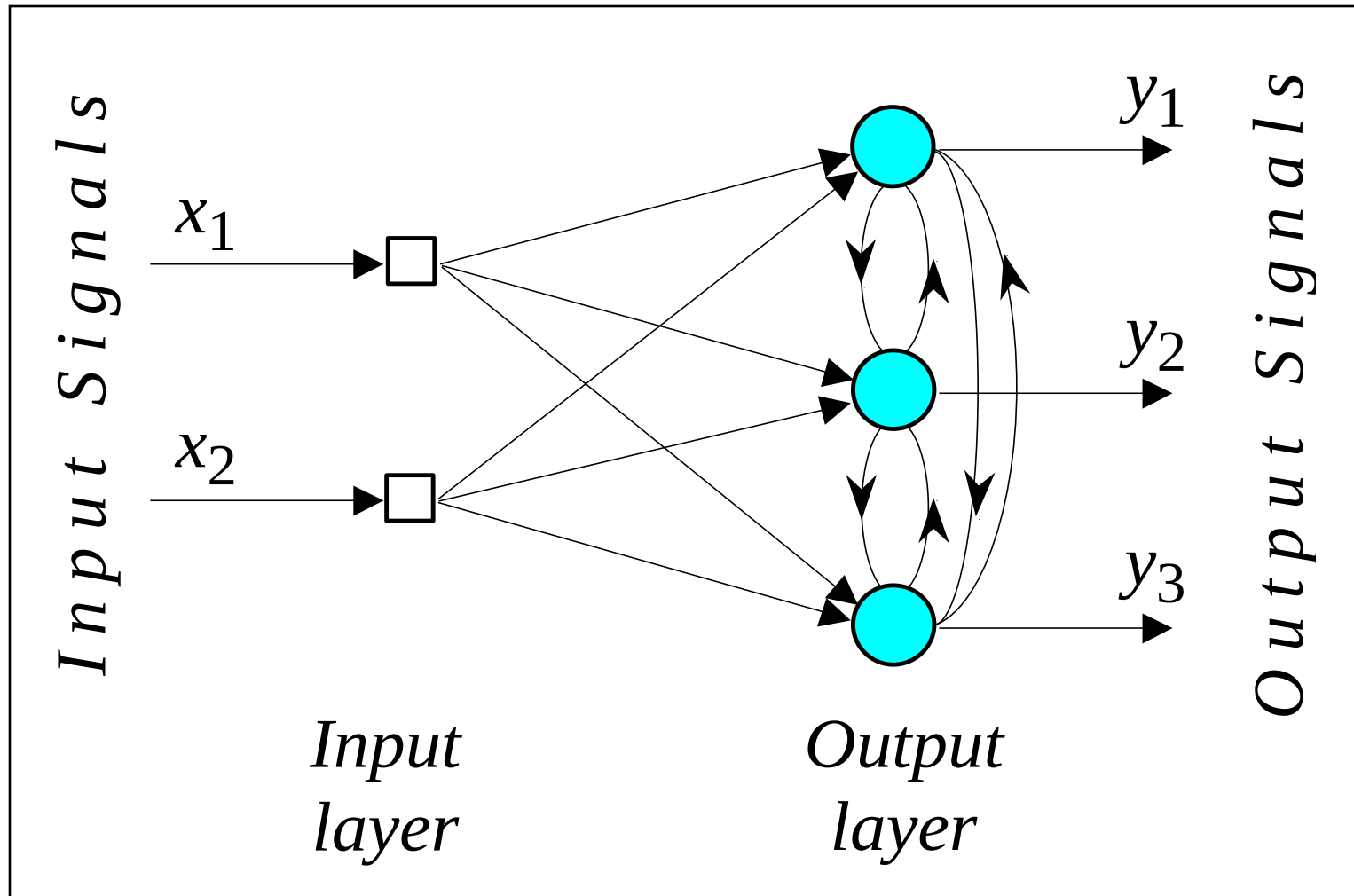




The Kohonen network

- The Kohonen model provides a topological mapping. It places a fixed number of input patterns from the input layer into a higher-dimensional output or Kohonen layer.
- Training in the Kohonen network begins with the winner's neighbourhood of a fairly large size. Then, as training proceeds, the neighbourhood size gradually decreases.

Architecture of the Kohonen Network

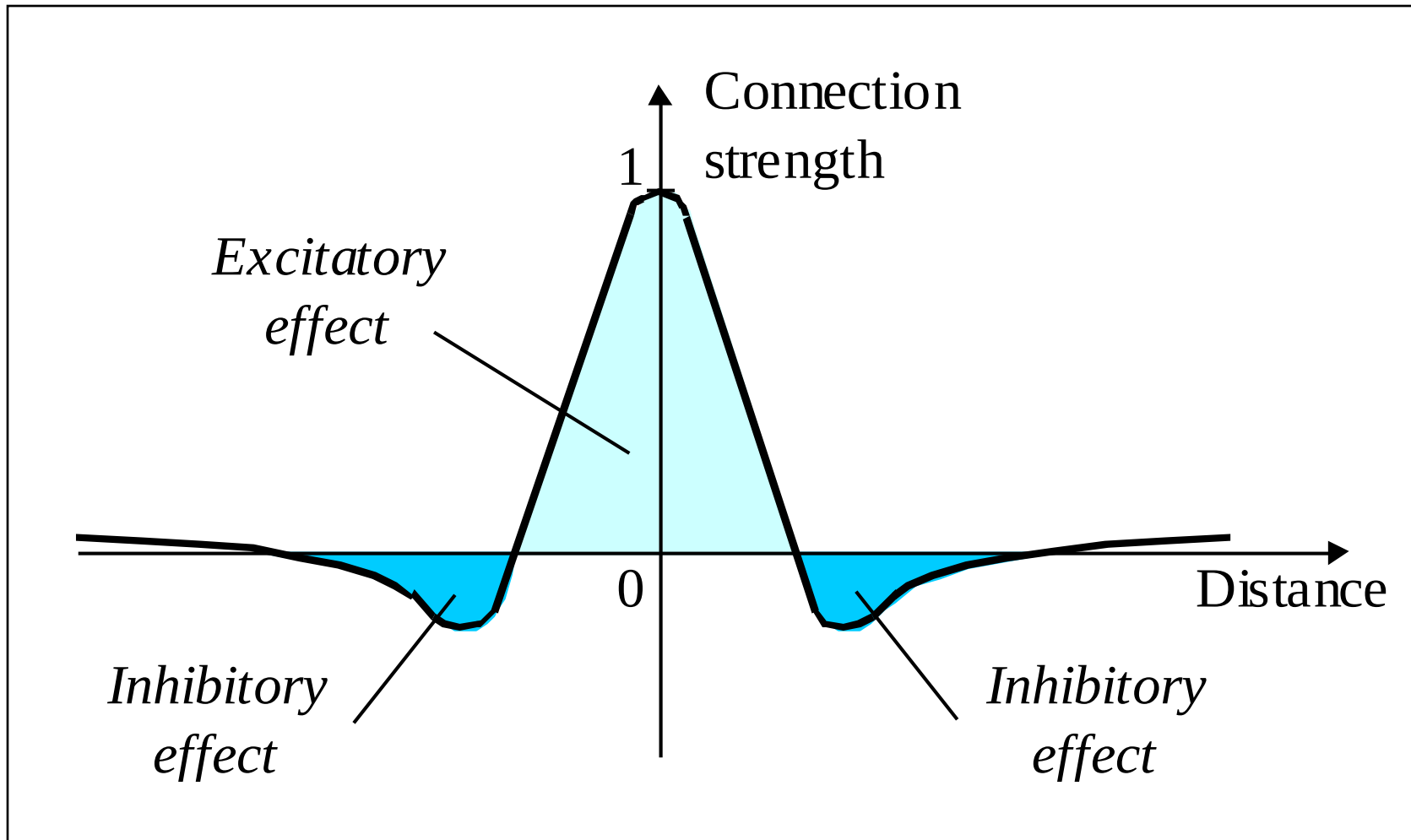




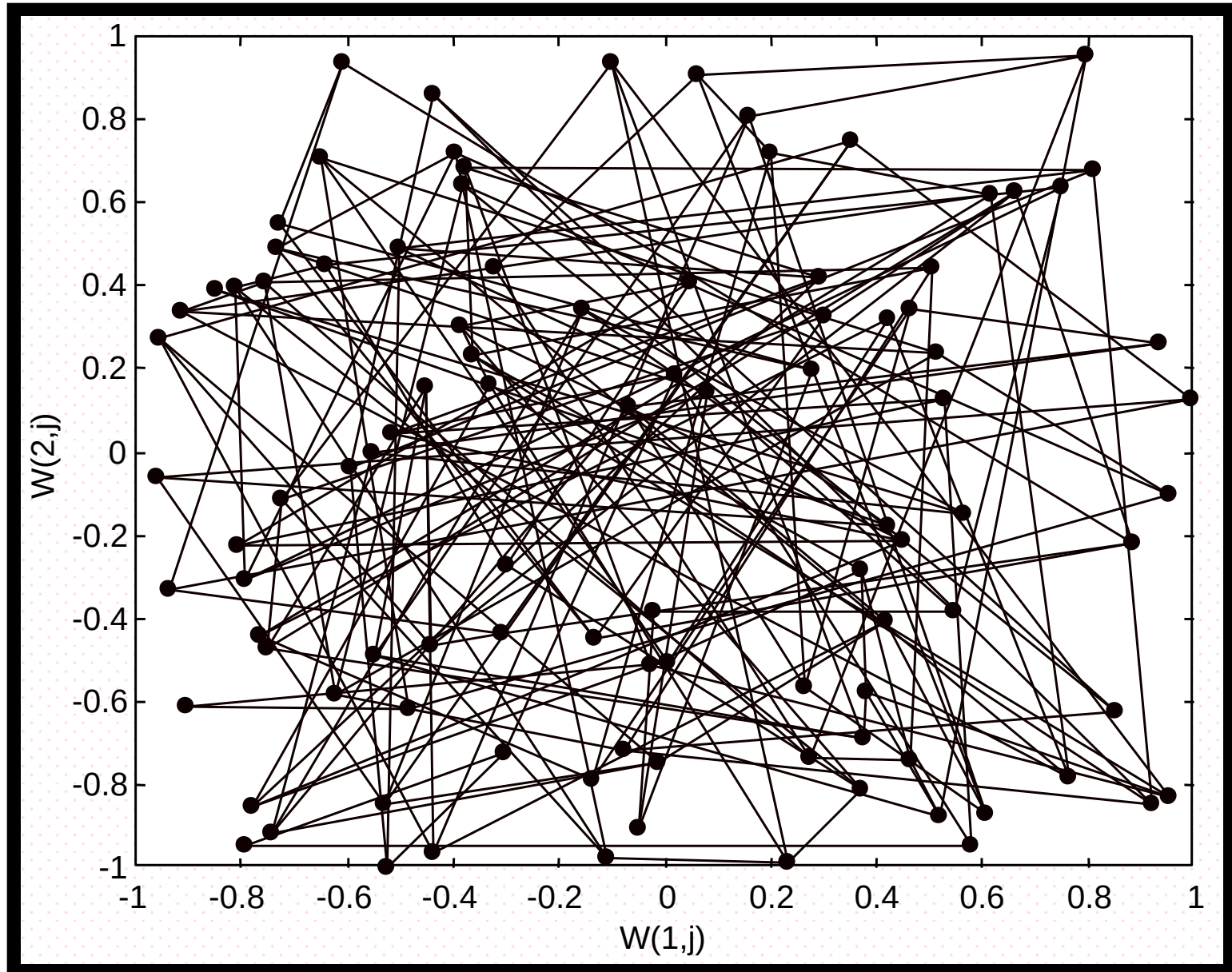
The Kohonen network

- The lateral connections are used to create a competition between neurons. The neuron with the largest activation level among all neurons in the output layer becomes the winner. This neuron is the only neuron that produces an output signal. The activity of all other neurons is suppressed in the competition.
- The lateral feedback connections produce excitatory or inhibitory effects, depending on the distance from the winning neuron. This is achieved by the use of a **Mexican hat function** which describes synaptic weights between neurons in the Kohonen layer.

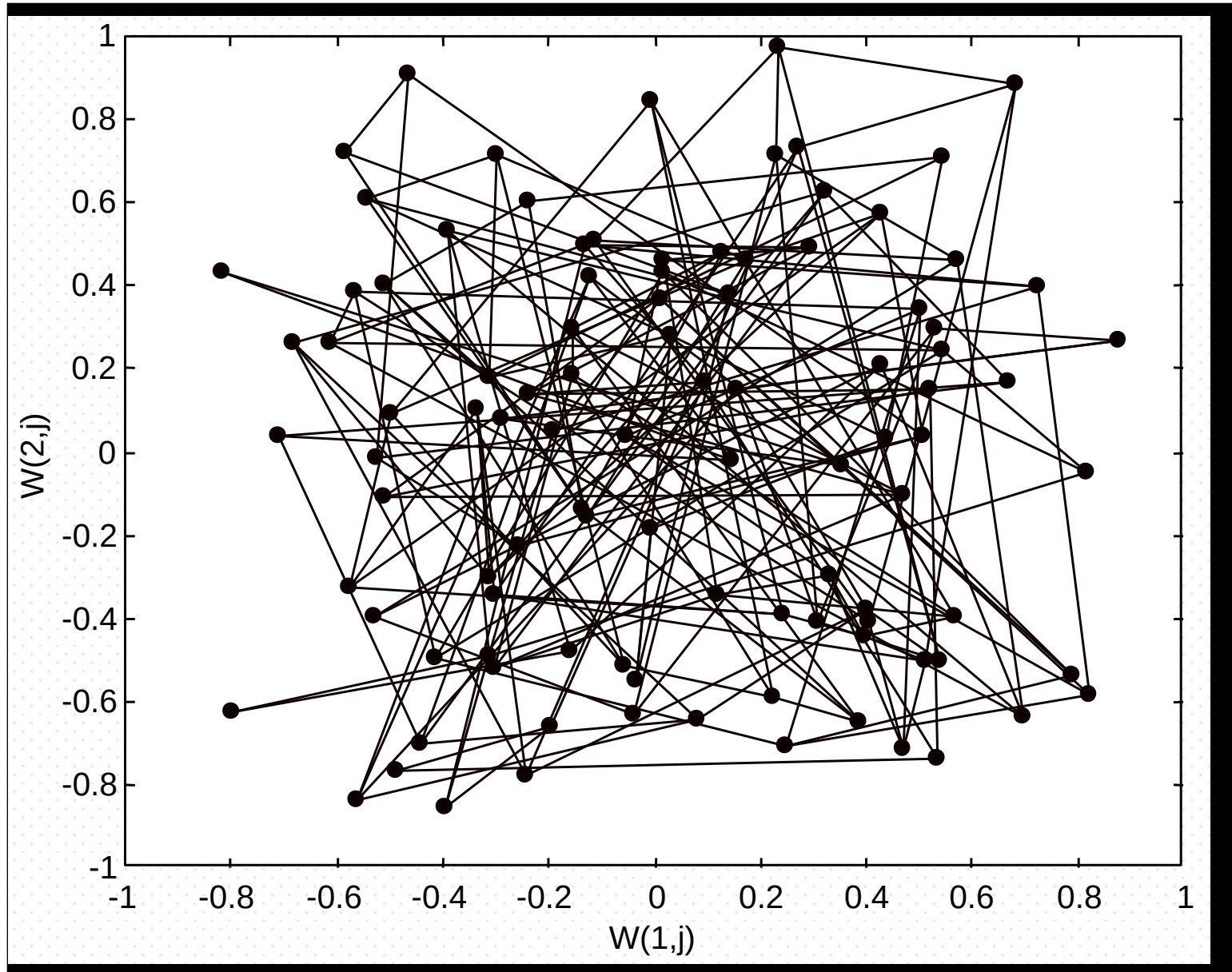
The Mexican hat function of lateral connection



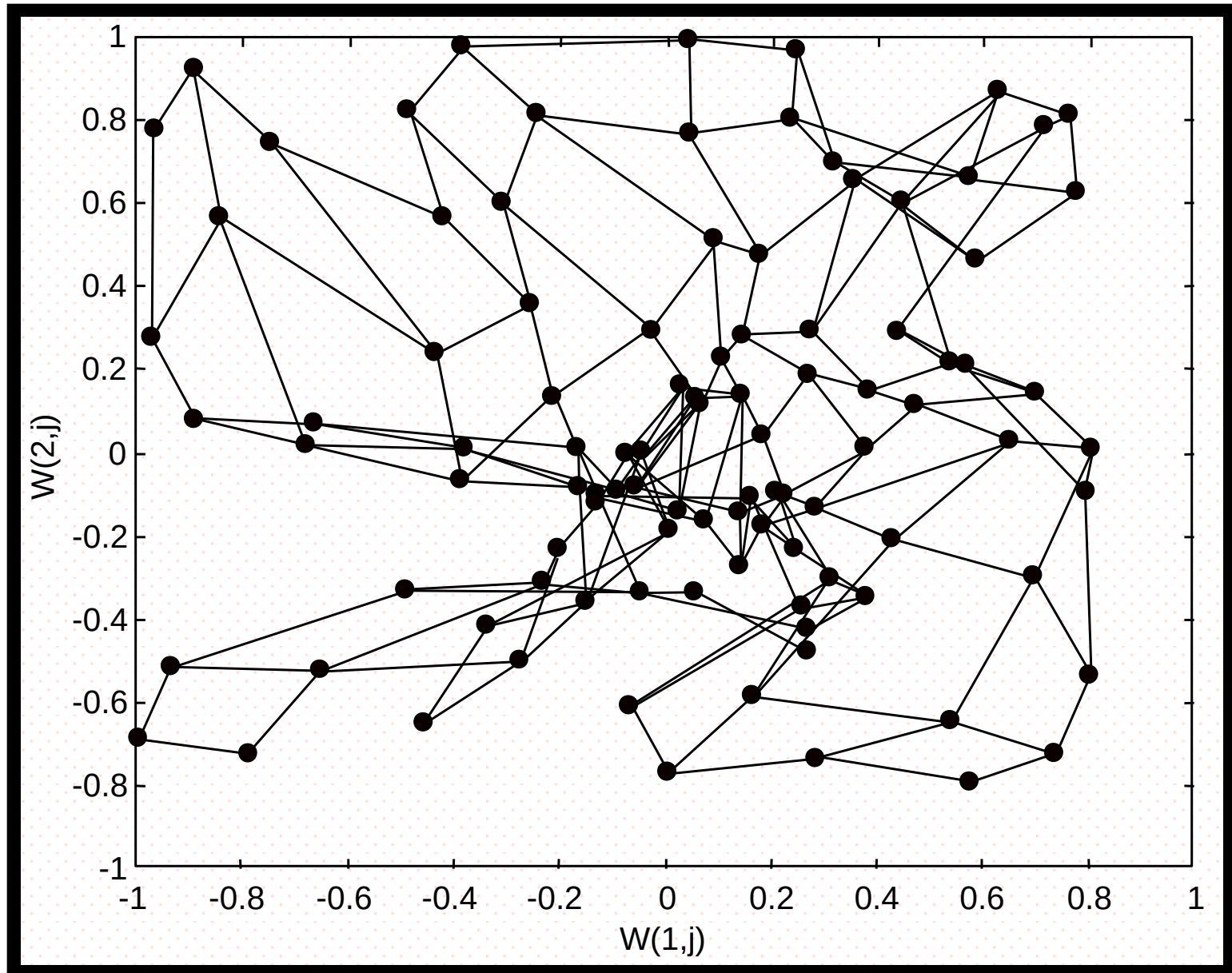
Initial random weights



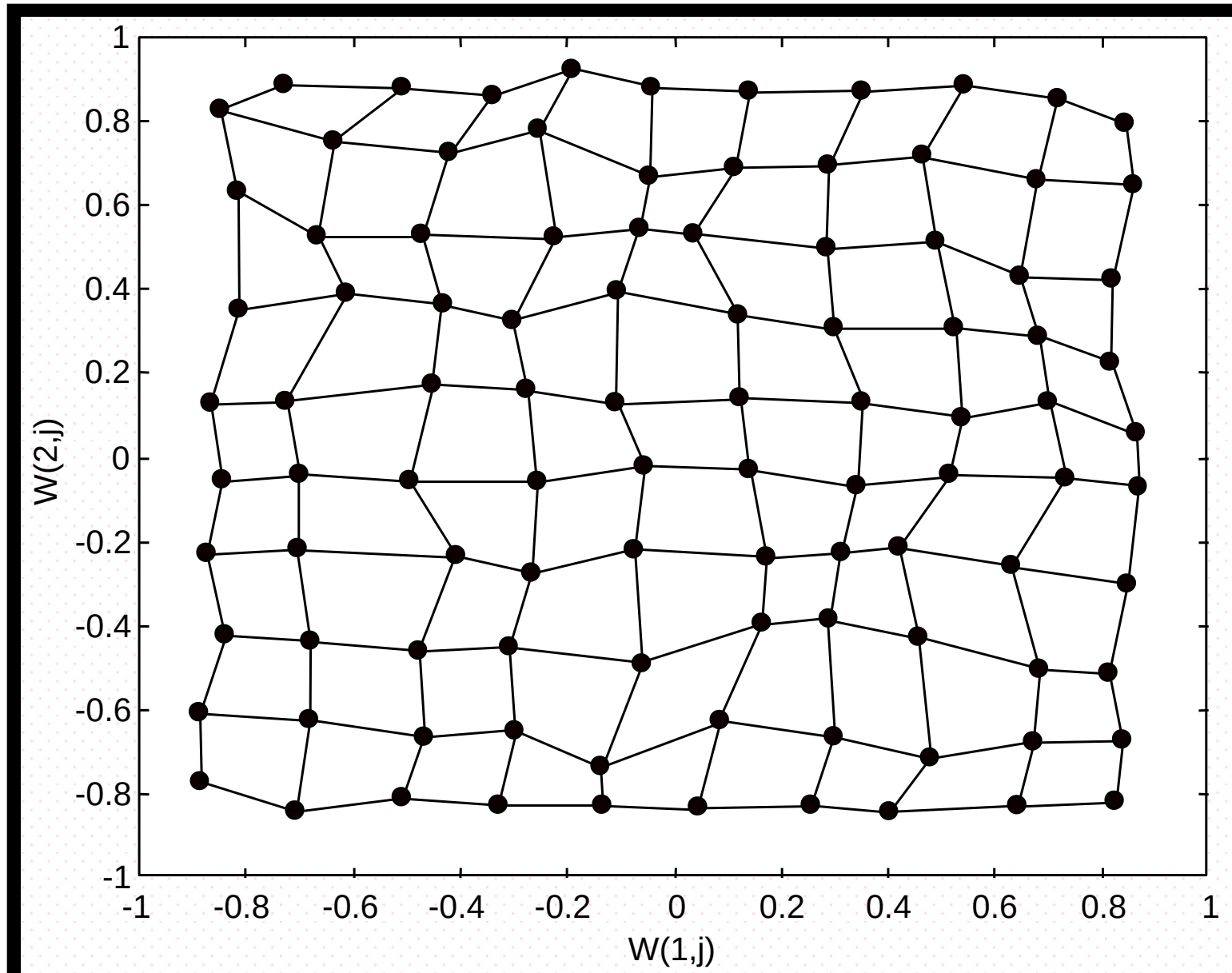
Network after 100 iterations



Network after 1000 iterations



Network after 10,000 iterations





An Example

- Suppose, for instance, that the 2-dimensional input vector $\mathbf{X}$ is presented to the three-neuron Kohonen network,

$$\mathbf{X} = \begin{bmatrix} 0.52 \\ 0.12 \end{bmatrix}$$

- The initial weight vectors, $\mathbf{W}_j$, are given by

$$\mathbf{W}_1 = \begin{bmatrix} 0.27 \\ 0.81 \end{bmatrix} \quad \mathbf{W}_2 = \begin{bmatrix} 0.42 \\ 0.70 \end{bmatrix} \quad \mathbf{W}_3 = \begin{bmatrix} 0.43 \\ 0.21 \end{bmatrix}$$



Kohonen network

- We find the winning (best-matching) neuron j_x using the minimum-distance Euclidean criterion:

$$d_1 = \sqrt{(x_1 - w_{11})^2 + (x_2 - w_{21})^2} = \sqrt{(0.52 - 0.27)^2 + (0.12 - 0.81)^2} = 0.73$$

$$d_2 = \sqrt{(x_1 - w_{12})^2 + (x_2 - w_{22})^2} = \sqrt{(0.52 - 0.42)^2 + (0.12 - 0.70)^2} = 0.59$$

$$d_3 = \sqrt{(x_1 - w_{13})^2 + (x_2 - w_{23})^2} = \sqrt{(0.52 - 0.43)^2 + (0.12 - 0.21)^2} = 0.13$$

- Neuron 3 is the winner and its weight vector $\mathbf{W}_3$ is updated according to the competitive learning rule.

$$\Delta w_{13} = \alpha (x_1 - w_{13}) = 0.1 (0.52 - 0.43) = 0.01$$

$$\Delta w_{23} = \alpha (x_2 - w_{23}) = 0.1 (0.12 - 0.21) = -0.01$$



Kohonen network

- The updated weight vector $\mathbf{W}_3$ at iteration $(p + 1)$ is determined as:

$$\mathbf{W}_3(p + 1) = \mathbf{W}_3(p) + \Delta\mathbf{W}_3(p) = \begin{bmatrix} 0.43 \\ 0.21 \end{bmatrix} + \begin{bmatrix} 0.01 \\ -0.01 \end{bmatrix} = \begin{bmatrix} 0.44 \\ 0.20 \end{bmatrix}$$

- The weight vector $\mathbf{W}_3$ of the winning neuron 3 becomes closer to the input vector $\mathbf{X}$ with each iteration.

Adaptive Resonance Theory (ART)



- **Adaptive Resonance Theory (ART)** is a cognitive and neural network theory that explains how the brain processes information in a stable yet flexible manner—able to learn new information without forgetting previously learned knowledge. It was introduced by **Stephen Grossberg** and **Gail Carpenter** in the 1980s.

- **Core Idea**
ART is designed to solve the **stability-plasticity dilemma** in learning systems:
 - **Stability**: Retaining previously learned knowledge.
 - **Plasticity**: Being able to learn new information quickly.Traditional neural networks often struggle with this balance. ART models solve it using **resonance**, a feedback loop between input and memory.

Adaptive Resonance Theory (ART)



How It Works?

Input Pattern: An input vector is presented to the network.

Comparison: The input is compared to existing memory patterns (categories).

Resonance Check:

If the input is **similar enough** to an existing pattern, it resonates.

If not, a **new category** is formed.

Vigilance Parameter: Controls the similarity threshold.

High vigilance = more specific categories (fine distinctions).

Low vigilance = broader categories (generalization).

Fuzzy ART Classification Example



Suppose we have the following input vectors representing objects (simplified for illustration):

Input Vector	Meaning
A = [0.9, 0.1]	Mostly Red object
B = [0.8, 0.2]	Another Red object
C = [0.1, 0.9]	Mostly Blue object

Key Parameters

Vigilance parameter (ρ) = 0.8

(Controls how similar new inputs must be to existing patterns.)

Categories: Start with none.

Fuzzy ART Classification Example



Step-by-Step Classification

Input A = [0.9, 0.1]

No existing categories → Create **Category 1**.

Store A as prototype for Category 1.

Input B = [0.8, 0.2]

Compare with Category 1 → similarity is high.

If **match** $\geq$ **ρ (0.8)** → **resonance occurs**.

Update Category 1 with new prototype:

New prototype = $\min([0.9, 0.1], [0.8, 0.2]) = [0.8, 0.1]$

Input C = [0.1, 0.9]

Compare with Category 1 → match is low.

No resonance → Create **Category 2**, store C.

Final Categories

Category 1: Red-ish objects → [0.8, 0.1]

Category 2: Blue-ish objects → [0.1, 0.9]

Adaptive Resonance Theory (ART)



3. Match Score Computation

A similarity score (usually the **fuzzy AND** operation or **dot product**, depending on the ART variant) is computed.

Example (Fuzzy ART match function):

$$\text{Match} = \frac{||I \wedge W_j||}{||I||}$$

Where:

- I is the input vector
- W_j is the j -th category prototype
- $\wedge$ is the fuzzy AND (min of each component)
- $|| \cdot ||$ is the norm (e.g., L1 norm)

Adaptive Resonance Theory (ART)



4. Vigilance Test

The match is compared to a threshold called **vigilance** (denoted ρ , between 0 and 1):

If **Match $\geq \rho$ $\rightarrow$ Resonance occurs**

If **Match $< \rho$ $\rightarrow$ Mismatch reset**: current category is inhibited, and the search continues.

5. Resonance State

Once a match passes the vigilance test, the following happens:

The category “**resonates**” with the input.

A **feedback loop** forms between input and the matched category.

The system becomes **temporarily stable**.

Learning occurs: the prototype vector is adjusted to include the new pattern.

This is like the brain saying: “Yes! This new experience fits well with what I already know—let’s update that memory.”

Adaptive Resonance Theory (ART)



In the context of ART, **resonance** is a **state of match and agreement** between the incoming input pattern and a stored memory (category) in the recognition layer. When this match is good enough (as judged by the **vigilance parameter**), the system enters a resonant state, allowing **learning to occur**.

The Resonance Process (Step-by-Step)

1. Input Pattern Presented

An input vector (e.g., $[0.9, 0.1]$) is introduced into the system.

2. Comparison with Stored Categories

The input is compared to each category (prototype vector) stored in the recognition layer.

Adaptive Resonance Theory (ART)



In ART, **temporary stability** refers to a brief moment during which the **interaction between the input and a matching category becomes locked-in**—meaning:

- The system **recognizes** the input.
- The **resonance** confirms that this input fits an existing memory (category).
- No further search is required.
- The **learning process is safely activated**.

Adaptive Resonance Theory (ART)



Why "Temporary"?

Because the system:

- Is always **open to new inputs**.
- Must remain **adaptable**.
- Will only stay stable **as long as the current input matches** a stored pattern.

Once a new input arrives:

- It could match the same category → stability continues.
- Or it may not match any existing category → **instability returns**, triggering search and reset mechanisms.

So this **stability is temporary**, not permanent—it only holds during successful resonance with the current input.

Adaptive Resonance Theory (ART)



Technical Meaning

In terms of computation:

- During **resonance**, a **feedback loop** forms between the comparison and recognition layers.
- This feedback loop stabilizes the activity in the network.
- No nodes are being suppressed or reset during this phase.
- The system enters a calm state → ready for **learning**.

Once the resonance ends:

- The loop breaks.
- The system becomes unstable again and ready to respond to the next input.

Adaptive Resonance Theory (ART)



Analogy: Jigsaw Puzzle

Imagine you're solving a jigsaw puzzle:

- You pick up a piece (input).
- You test it in different spots (compare with categories).
- One spot fits perfectly (resonance).
- You press the piece in → moment of stability (learning).
- Now you pick the next piece → the process begins again (instability returns).

That **moment when a piece fits** and you stop looking is like the **temporary stability** in ART.

Adaptive Resonance Theory (ART)



Short-Term Memory (STM) is:

A **temporary storage** of the **current input pattern**, used for comparison and matching with stored categories (long-term memory).

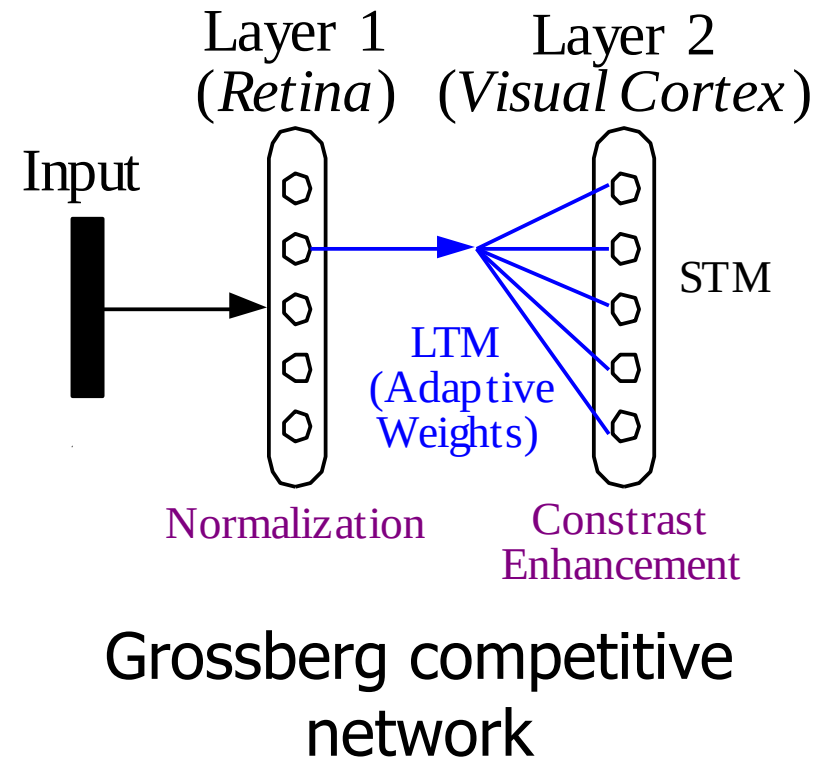
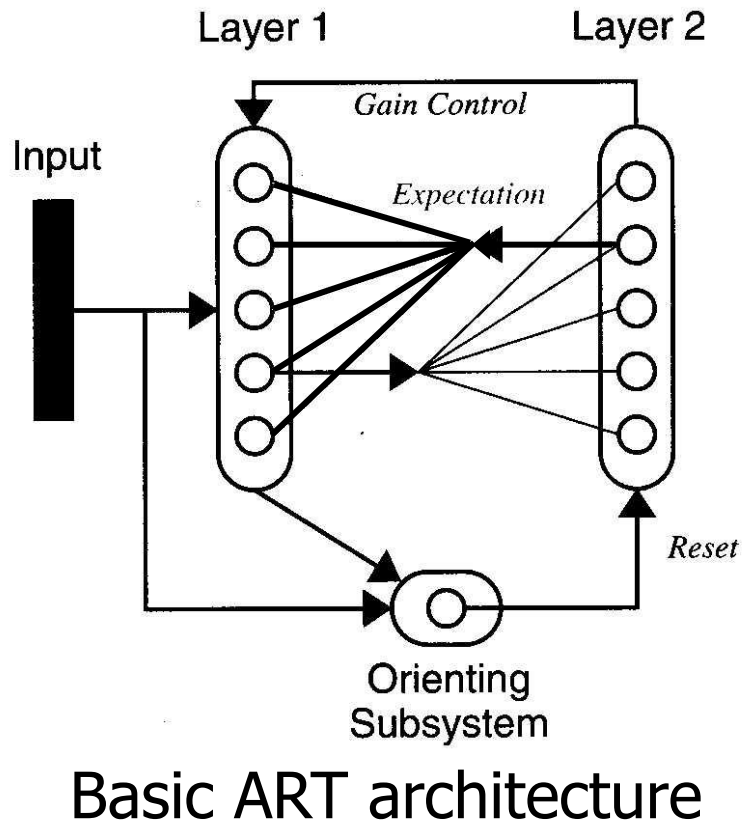
Where It Happens:

In ART models, the **F₁ layer (input layer)** holds the input pattern in **short-term memory**, while the **F₂ layer** holds **long-term memory (LTM)** in the form of category prototypes.

Feature	Short-Term Memory (STM)	Long-Term Memory (LTM)
Location	F ₁ Layer (input layer)	F ₂ Layer (recognition/category layer)
Function	Holds current input	Stores learned categories
Duration	Active during a single input cycle	Persistent across multiple inputs
Learns?	No	Yes (updated after resonance)



Overview





Grossberg Network

- The L1-L2 connections are **instars**, which performs a **clustering** (or **categorization**) operation. When an input pattern is presented, it is multiplied (after normalization) by the L1-L2 weight matrix.
- A **competition** is performed at Layer 2 to determine which row of the weight matrix is closest to the input vector. That row is then moved toward the input vector.
- After learning is complete, **each row of the L1-L2 weight matrix is a prototype pattern**, which represents a cluster (or a category) of input vectors.



ART Networks

- Learning of ART networks also occurs in a set of **feedback connections** from Layer 2 to Layer 1. These connections are **outstars** which perform **pattern recall**.
- When a node in **Layer 2** is activated, this **reproduces a prototype pattern** (the expectation) at layer 1.
- **Layer 1** then performs a **comparison** between the **expectation** and the **input pattern**.
- When the expectation and the input pattern are **NOT** closely matched, the **orienting subsystem** causes a **reset** in Layer 2.



ART Networks

- The **reset disables** the **current** winning neuron, and the current expectation is removed.
- A **new competition** is then performed in Layer 2, while the previous winning neuron is **disable**.
- The new winning neuron in Layer 2 projects a new expectation to Layer 1, through the L2-L1 connections.
- This process continues until **the L2-L1 expectation provides a close enough match to the input pattern.**



ART Subsystems

Layer 1

Comparison of input pattern and expectation.

L1-L2 Connections (Instars)

Perform clustering operation.

Each *row* of $W^{1:2}$ is a prototype pattern.

Layer 2

Competition (Contrast enhancement)

L2-L1 Connections (Outstars)

Perform pattern recall (Expectation).

Each *column* of $W^{2:1}$ is a prototype pattern

Orienting Subsystem

Causes a reset when expectation does not match input pattern

Disables current winning neuron



Thank you

Questions?